# UCab — Full-Stack MERN Cab Booking Platform

Solo Developer / Team Member: Shaik Sumiya Zainab | Project ID: N/A (Solo Track)

# Phase 1: Brainstorming & Ideation — Define Problem Statements

Project Name: Cab Booking (`UCab`)

Project ID: `N/A (Solo Track Submission)`

Team Member / Solo Developer: Shaik Sumiya Zainab

## 1. Background & Industry Context

Urban mobility is one of the most critical pillars of modern city infrastructure. Millions of commuters rely daily on on-demand cab services for work commutes, airport transfers, medical appointments, and leisure travel. However, existing commercial ride-booking solutions often suffer from systemic issues: lack of transparency in fare breakdowns, hidden surge charges, complex user interfaces that deter non-technical users, and rigid booking options that do not cater to personalized rider comfort.

From an administrative and fleet management perspective, independent taxi operators and small-to-medium dispatch businesses lack affordable, turnkey web software that allows them to track vehicles, assign drivers, and monitor revenue in real time without heavy monthly subscription fees or complex database maintenance.

## 2. Core Problem Statements

### Problem Statement 1: Pricing Ambiguity & Lack of Fare Transparency

• The Problem: Commuters frequently experience price shock due to opaque fare calculation algorithms. When selecting different vehicle categories (e.g., Mini, Sedan, SUV, Luxury), users are not shown the exact per-kilometer rate, base fare, or how distance impacts the total price before booking.

• Impact: Reduced user trust, high trip cancellation rates at the last minute, and customer dissatisfaction.

• Our Solution (`UCab`): Implement a real-time, deterministic fare calculation engine right on the booking screen (`Base Fare + (Distance in KM * Price Per KM) - Discount`). Every cab type explicitly displays its base fare (` ¹40` to ` ¹150`) and per-km rate (` ¹10/km` to ` ¹35/km`), ensuring total price transparency.

### Problem Statement 2: Absence of Personalized & Human-Centric Ride Perks

• The Problem: Most traditional cab platforms treat rides purely as transactional movement from Point A to Point B. Riders have no mechanism to request in-ride hospitality (such as bottled drinking water or snacks) prior to pickup, nor can they contribute directly to social welfare causes related to drivers.

• Impact: Monotonous user experience and missed revenue opportunities for value-added services.

• Our Solution (`UCab`): Introduce an innovative In-Ride Customization & Charity Suite:

• Refreshments Add-On (`+ ¹50`): Pre-orders chilled mineral water and premium snacks waiting inside the vehicle upon pickup.

• Road Safety & Driver Welfare Donation (`+ ¹20`): Allows socially conscious commuters to make a direct round-up contribution to driver emergency healthcare funds.

### Problem Statement 3: High Configuration Friction & Database Dependency in Development/Evaluation

• The Problem: When deploying, evaluating, or demonstrating full-stack web applications, external dependencies (like remote MongoDB clusters or local database daemons) often fail due to network restrictions, missing environment variables, or port conflicts (`ECONNREFUSED`).

• Impact: Applications crash on startup, making live demonstrations or offline grading impossible.

• Our Solution (`UCab`): Build a resilient Zero-Config Dual-Engine Database Architecture. The Express.js backend checks for an active MongoDB instance at startup. If unreachable, it automatically activates an atomic, file-backed JSON engine (`server/db/data.json`) that transparently supports all Mongoose queries without code changes or downtime.

## 3. Solo Developer Execution Strategy

As a solo developer handling the entire product lifecycle (UX Research, UI Design, Frontend Engineering, Backend API Development, Database Design, and QA Testing), the focus was placed on:

1. Clean Modular Architecture: Separating React presentation layers cleanly from Express controller logic.

2. Component Reusability: Designing unified cards (`CabListing`, `BookCab`, `ReceiptModal`) to maintain visual consistency.

3. Automated Fallbacks: Eliminating external infrastructure blockers through self-seeding storage engines.